6. OpenClaw Tools Introduction

The tools are the "capability interfaces" that OpenClaw exposes to the agent; they let the model carry out real operations in a safer and more structured way, such as reading and writing files, running commands, managing the sessions, browsing the web and the UI automation. Compared with the old style `openclaw-*` Skills, the tools are typed, controllable and they can be restricted precisely through the configuration to say what it is allowed to do and what it is not.

6. OpenClaw Tools Introduction

1. The Key Concepts
   - 1.1 The Relation Between the Tools and the Skills
   - 1.2 More Tools Is Not Better
   - 1.3 The Core of the Tool Policy: profile + allow/deny
2. Quick Start
   - 2.1 Choose a Tool Configuration File (profile)
   - 2.2 Control It Precisely with allow/deny (it is recommended to start from deny)
   - 2.3 Tighten It for One Provider/Model Only (byProvider)
3. The Tool Groups (group:*)
4. The High Risk Tools: How to Configure Them Safely
   - 4.1 The Command Execution Ones (group:runtime)
   - 4.2 A Note About apply_patch
   - 4.3 Isolate the Tools per Agent (the recommended approach)
5. The Recommended Configuration Templates (ready to use)
   - 5.1 The Daily Conversation (safer)
   - 5.2 Learning / Writing Documentation (browsing is allowed, running commands is not)
   - 5.3 Development and Debugging (reading and writing the files is allowed, but treat running commands as an "option")
6. Frequently Asked Questions
   - 6.1 The allow Is Configured but It Does Not Take Effect
   - 6.2 A Certain Model Can Use the Tools, but It Does Not Work After Switching the Model
   - 6.3 Which Profile Should I Choose?
7. The Official Documentation

1. The Key Concepts

1.1 The Relation Between the Tools and the Skills

- Tools: the system level "capability interfaces"; they are typed, configurable and they can be restricted per provider/model
- Skills: more oriented towards the "reusable workflow/plugin capability"; many of the old skills are replaced or weakened by the tool system

If you want the agent to "do something", think about the tools first; if you want to encapsulate a flow into a reusable capability, then think about the Skills.

1.2 More Tools Is Not Better

The more tools there are, the more the model can do, but the bigger the risk as well. The best practice is: enable by default only the tools that you really need, and switch the high risk capabilities (especially running commands) off separately or enable them on demand.

1.3 The Core of the Tool Policy: profile + allow/deny

The tool set is usually trimmed in this order:

1. `tools.profile` first gives a "basic allow set"
2. If `tools.byProvider` is configured, it further restricts a given provider or provider/model (it can only shrink the tool set)
3. Finally `tools.allow` / `tools.deny` restrict or open it precisely (`deny` has the priority)

2. Quick Start

The tool allow policy of OpenClaw is written in `openclaw.json`. In general you do not have to configure it manually

- `deny` has a higher priority than `allow`
- The `*` wildcard is supported, and the matching is not case sensitive

2.1 Choose a Tool Configuration File (profile)

The common profiles (ordered by "from the fewest capabilities to the most"):

- `minimal` : only `session_status`
- `messaging` : oriented towards the messaging capabilities + the sessions capabilities
- `coding` : oriented towards the development/engineering capabilities (the files, the runtime, the sessions, the memory, the images and so on)
- `full` : no restriction (the same as not setting it)

Example: enable the coding profile

```
{
  "tools": {
    "profile": "coding"
  }
}
```

2.2 Control It Precisely with allow/deny (it is recommended to start from deny)

Example: disable the `browser` tool globally

```
{
  "tools": {
    "deny": ["browser"]
  }
}
```

Example: on the basis of coding, disable all the runtime tools (exec / bash / process)

```
{
  "tools": {
    "profile": "coding",
    "deny": ["group:runtime"]
  }
}
```

2.3 Tighten It for One Provider/Model Only (byProvider)

You can keep the global default unchanged and only tighten the tool set further for a given provider or provider/model (it can only shrink it, never expand it).

Example: use coding globally, but minimal for Google Antigravity

```
{
  "tools": {
    "profile": "coding",
    "byProvider": {
      "google-antigravity": { "profile": "minimal" }
    }
  }
}
```

Example: reduce the tools further for `openai/gpt-5.2`

```
{
  "tools": {
    "allow": ["group:fs", "group:runtime", "sessions_list"],
    "byProvider": {
      "openai/gpt-5.2": { "allow": ["group:fs", "sessions_list"] }
    }
  }
}
```

3. The Tool Groups (group:*)

In `tools.allow` / `tools.deny` you can control a group of tools at once with `group:*`:

Group name	Expands to
`group:runtime`	exec, bash, process

Group name	Expands to
`group:fs`	read, write, edit, apply_patch
`group:sessions`	sessions_list, sessions_history, sessions_send, sessions_spawn, session_status
`group:memory`	memory_search, memory_get
`group:web`	web_search, web_fetch
`group:ui`	browser, canvas
`group:automation`	cron, gateway
`group:messaging`	message
`group:nodes`	nodes
`group:openclaw`	all the built in OpenClaw tools (the provider plugins are not included)

Example: only allow the file tools + browser

```
{
  "tools": {
    "allow": ["group:fs", "browser"]
  }
}
```

4. The High Risk Tools: How to Configure Them Safely

4.1 The Command Execution Ones (group:runtime)

`group:runtime` contains the execution capabilities (for example exec / bash / process). It is recommended to follow these rules for this kind of capability:

- Disabled by default: start with `deny: ["group:runtime"]` and open it up step by step when it is needed
- Open it only for the "specific agents": concentrate the high risk capabilities on a dedicated agent
- Open it only for the "specific provider/model": give more tools to the stable/trusted models

4.2 A Note About apply_patch

`apply_patch` belongs to the file editing tools; in some of the versions it is an experimental feature and it may have to be enabled with an extra switch (for example through `tools.exec.applyPatch.enabled`, and only some of the models support it). If you find that the model clearly knows how to modify a file but simply cannot call apply_patch, check the tool policy and the related switches first.

4.3 Isolate the Tools per Agent (the recommended approach)

You can set up one tool policy globally and then override it separately for a certain agent, so that the "high privilege capabilities" are concentrated on a few agents.

Example: coding globally, but the support agent only uses messaging and is additionally allowed the Slack tools

```
{
  "tools": { "profile": "coding" },
  "agents": {
    "list": [
      {
        "id": "support",
        "tools": { "profile": "messaging", "allow": ["slack"] }
      }
    ]
  }
}
```

5. The Recommended Configuration Templates (ready to use)

5.1 The Daily Conversation (safer)

Goal: only chat + only look at the session status, and do not give it the ability to "mess with the system".

```
{
  "tools": {
    "profile": "minimal"
  }
}
```

5.2 Learning / Writing Documentation (browsing is allowed, running commands is not)

```
{
  "tools": {
    "profile": "coding",
    "deny": ["group:runtime"],
    "allow": ["group:web"]
  }
}
```

5.3 Development and Debugging (reading and writing the files is allowed, but treat running commands as an "option")

```
{
  "tools": {
    "profile": "coding",
    "deny": ["group:runtime"]
  }
}
```

When you really have to execute the commands, open `group:runtime` temporarily and close it again afterwards.

6. Frequently Asked Questions

6.1 The allow Is Configured but It Does Not Take Effect

- Check whether `deny` also hits the same tool (deny has the priority)
- Check whether the name of the tool is spelled correctly (the matching is not case sensitive, but the name has to exist)
- If `tools.allow` only lists the plugin tools that are unknown or not loaded, the system may ignore the allow list in order to keep the core tools usable

6.2 A Certain Model Can Use the Tools, but It Does Not Work After Switching the Model

- You may have set `tools.byProvider` (the provider or the provider/model) to add an extra restriction for that model
- Some of the provider endpoints are not consistent in their support of the tools; it is recommended to use a smaller tool set for the unstable endpoints

6.3 Which Profile Should I Choose?

- You only want to chat: `minimal`
- The daily messaging + managing the sessions: `messaging`
- The file/engineering tasks: `coding`
- You definitely know what you are doing and you can bear the risk, and I want the complete and powerful OpenClaw: `full`

7. The Official Documentation

- The OpenClaw tools documentation: <Overview - OpenClaw>